Agenda

- clone()
- Date/Calender/LocalDate
- Strings
    - String
    - StringBuffer
    - StringBuilder
- String Tokenizer
- Enum

Clone method

- The clone() method is used to create a copy of an object in Java.
- It's defined in the java.lang.Object class and is inherited by all classes in Java.
- It returns a shallow copy of the object on which it's called.

```
protected Object clone() throws CloneNotSupportedException
```

- This means that it creates a new object with the same field values as the original object, but the fields themselves are not cloned.
- If the fields are reference types, the new object will refer to the same objects as the original object.
- In order to use the clone() method, the class of the object being cloned must implement the Cloneable interface.
- This interface acts as a marker interface, indicating to the JVM that the class supports cloning.
- It's recommended to override the clone() method in the class being cloned to provide proper cloning behavior.
- The overridden method should call super.clone() to create the initial shallow copy, and then perform any necessary deep copying if required.
- The clone() method throws a CloneNotSupportedException if the class being cloned does not implement Cloneable, or if it's overridden to throw the exception explicitly.

Cloneable interface

- Enable creating copy/clone of the object.
- If a class is Cloneable, Object.clone() method creates a shallow copy of the object.
- If class is not Cloneable, Object.clone() throws CloneNotSupportedException.
- A class should implement Cloneable and override clone() to create a deep/shallow copy of the object.

Date/ LocalDate/ Calender

- Date and Calender class are in java.util package
- The class Date represents a specific instant in time, with millisecond precision.
- the formatting and parsing of date strings were not standardized it is not recommended to use
- As of JDK 1.1, the Calendar class should be used to convert between dates and time fields and the DateFormat class should be used to format and parse date strings.

- LocalDate is in java.time Package
- It is immutable and threadsafe class.

Java DateTime APIs

- DateTime APIs till Java 7

```
// java.util.Date
Date d = new Date();
System.out.println("Timestamp: " + d.getTime());
// number of milliseconds since 1-1-1970 00:00.
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
System.out.println("Date: " + sdf.format(d));

// java.util.Date
String str = "28-09-1983";
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
Date d = sdf.parse(str);
System.out.println(d.toString());

// java.util.Calendar
Calendar c = Calendar.getInstance();
System.out.println(c.toString());
System.out.println("Current Year: " + calendar.get(Calendar.YEAR));
System.out.println("Current Month: " + calendar.get(Calendar.MONTH));
System.out.println("Current Date: " + calendar.get(Calendar.DATE));
```

- Limitations of existing DateTime APIs
  - Thread safety
  - API design and ease of understanding
  - ZonedDate and Time
- Most commonly used java 8 onwards new classes are LocalDate, LocalTime and LocalDateTime.
- LocalDate

```
LocalDate localDate = LocalDate.now();
LocalDate tomorrow = localDate.plusDays(1);
DayOfWeek day = tomorrow.getDayOfWeek();
int date = tomorrow.getDayOfMonth();
System.out.println("Date: " +
tomorrow.format(DateTimeFormatter.ofPattern("dd-MMM-yyyy")));

//LocalDate date = LocalDate.of(1983, 09, 28);
LocalDate d1 = LocalDate.parse("1983-09-28");
LocalDate d2 = LocalDate.parse("1983-09-28",
DateTimeFormatter.ofPattern("dd-MM-yyyy"));
System.out.println("Is Leap Year: " + d1.isLeapYear());
System.out.println("Is Leap Year: " + d2.isLeapYear());
```

- LocalTime

```
LocalTime now = LocalTime.now();
LocalTime nextHour = now.plus(1, ChronoUnit.HOURS);
System.out.println("Hour: " + nextHour.getHour());
System.out.println("Time: " +
nextHour.format(DateTimeFormatter.ofPattern("HH:mm")));
```

- LocalDateTime

```
LocalDateTime now = LocalDateTime.now();
LocalDateTime dt = LocalDateTime.parse("2000-01-30T06:30:00");
dt.minusHours(2);
System.out.println(dt.toString());
```

Strings

- java.lang.Character is wrapper class that represents char.
- In Java, each char is 2 bytes because it follows unicode encoding.
- String is sequence of characters.
    - 1. java.lang.String: "Immutable" character sequence
    - 2. java.lang.StringBuffer: Mutable character sequence (Thread-safe)
    - 3. java.lang.StringBuilder: Mutable character sequence (Not Thread-safe)
- String helpers
    - 1. java.util.StringTokenizer: Helper class to split strings

String Class Object

- java.lang.String is class and strings in java are objects.
- String constants/literals are stored in string pool.
- String objects created using "new" operator are allocated on heap.
- In java, String is immutable. If try to modify, it creates a new String object on heap.

```
String name = "sunbeam"; // goes in string pool

String name2 = new String("Sunbeam"); // goes on heap
```

- Since strings are immutable, string constants are not allocated multiple times.
- String constants/literals are stored in string pool. Multiple references may refer the same object in the pool.
- String pool is also called as String literal pool or String constant pool.
- From Java 7, String pool is in the heap space (of JVM).
- The string literal objects are created during class loading.

StringBuffer and StringBuilder

- StringBuffer and StringBuilder are final classes declared in java.lang package.
- It is used create to mutable string instance.
- equals() and hashCode() method is not overridden inside it.
- Can create instances of these classes using new operator only. Objects are created on heap.
- StringBuffer implementation is thread safe while StringBuilder is not thread-safe.
- StringBuilder is introduced in Java 5.0 for better performance in single threaded applications.
- The default capactiy is 16.
- If the capacity is full it increases i.e new capacity is allocated

```
newCapacity = (oldCapacity*2) + 2
```

String Tokenizer

- Used to break a string into multiple tokens - like split() method.
- Methods of java.util.StringTokenizer
    - boolean hasMoreTokens()
    - String nextToken()
    - String nextToken(String delim)

```

```

String str = "sunbeam infotech"; StringTokenizer tokenizer = new StringTokenizer(str);

```
```JAVA
String str = "www.sunbeaminfo.com";
String delim = ".";
StringTokenizer tokenizer = new StringTokenizer(str, delim);
```

```
String str = "https://admission.sunbeaminfo.com/placements";
String delim = "://.";
StringTokenizer tokenizer = new StringTokenizer(str, delim, true);

while (tokenizer.hasMoreTokens()) {
  String token = tokenizer.nextToken();
  System.out.println(token);
}
```

Enum

- In C enums were internally integers
```

- In java, It is a keyword added in java 5 and enums are object in java.
- used to make constants for code readability
- mostly used for switch cases
- In java, enums cannot be declared locally (within a method).
- The enums constants can be used in switch-case and can also be compared using == operator.

```
// Without Enum
public class Program01 {

public static int menu() {
    Scanner sc = new Scanner(System.in);
    System.out.println("0. EXIT");
    System.out.println("1. ADD");
    System.out.println("2. SUB");
    System.out.println("3. MUL");
    System.out.println("4. DIV");
    System.out.println("Enter your choice - ");
    return sc.nextInt();
}

public static void main(String[] args) {
  int choice;
  while ((choice = menu1()) != 0) {
      switch (choice) {
          case 1:
              System.out.println("Addition selected");
              break;
          case 2:
              System.out.println("Substraction selected");
              break;
          case 3:
              System.out.println("Multiplication selected");
              break;
          case 4:
              System.out.println("Division selected");
              break;
          case 5:
        }
    }
  }
}
```



```
// With enum
public class Program01 {

enum ECalculator {
    EXIT, ADD, SUB, MUL, DIV
}

public static ECalculator menu2() {
```

```
  Scanner sc = new Scanner(System.in);
  ECalculator arr[] = ECalculator.values();
  for (ECalculator e : arr)
      System.out.println(e.ordinal() + ". " + e.name());
  System.out.println("Enter your choice - ");
  return arr[sc.nextInt()];
}

public static void main(String[] args) {
    ECalculator choice;
    while ((choice = menu2()) != ECalculator.EXIT) {
        switch (choice) {
            case ADD:
                System.out.println("Addition selected");
                break;
            case SUB:
                System.out.println("Substraction selected");
                break;
            case MUL:
                System.out.println("Multiplication selected");
                break;
            case DIV:
                System.out.println("Division selected");
                break;
          }
      }
  }
}
```

- The enum type declared is implicitly inherited from java.lang.Enum class. So it cannot be extended from another class, but enum may implement interfaces.

```
public abstract class Enum<E> implements java.lang.Comparable<E>,
java.io.Serializable {
    private final String name;
    private final int ordinal;

    protected Enum(String,int); // sole constructor - can be called from user-
defined enum class only

    public final String name(); // name of enum const
    public final int ordinal(); // position of enum const (0-based)
    public String toString(); // returns name of const
    public final int compareTo(E);// compares with another enum of same type on
basis of ordinal number
    public static <T> T valueOf(Class<T>, String);
    // ...
}
```

- The declared enum is converted into enum class.

- The enum constants declared in enum are public static final fields of generated class.
- Enum objects cannot be created explicitly (as generated constructor is private).

```
// user-defined enum
enum ArithmeticOperations {
    ADDITION, SUBTRACTION, MULIPLICATION, DIVISION
}

// generated enum code
final class ArithmeticOperations extends Enum {

private ArithmeticOperations(String name, int ordinal) {
    super(name, ordinal); // invoke sole constructor Enum(String,int);
}

public static ArithmeticOperations[] values() {
    return (ArithmeticOperations[])$VALUES.clone();
}

public static ArithmeticOperations valueOf(String s) {
    return (ArithmeticOperations)Enum.valueOf(ArithmeticOperations,s);
}


  public static final ArithmeticOperations ADDITION;
  public static final ArithmeticOperations SUBTRACTION;
  public static final ArithmeticOperations MULIPLICATION;
  public static final ArithmeticOperations DIVISION;
  private static final ArithmeticOperations $VALUES[];

  static {
    ADDITION = new ArithmeticOperations("ADDITION", 0);
    SUBTRACTION = new ArithmeticOperations("SUBTRACTION", 1);
    MULIPLICATION = new ArithmeticOperations("MULIPLICATION", 2);
    DIVISION = new ArithmeticOperations("DIVISION", 3);
    $VALUES = (new ArithmeticOperations[] {
      ADDITION, SUBTRACTION, MULIPLICATION, DIVISION
      });
  }
}
```

- The enum may have fields and methods.

```
enum AccountType {
SAVINGS(2.5) {
  @Override
  public String toString() {
    return "Savings";
  }
},
```

```
  CURRENT(1.5) {
    @Override
    public String toString() {
      return "Current";
    }
  },
  DMAT(0) {
    @Override
    public String toString() {
      return "Demat";
    }
  };

  private double roi;

  private AccountType3(double roi) {
    this.roi = roi;
  }

  public double getRoi() {
    return roi;
  }

  }
```